SPURVANCE MINI- PROJECT REPORT

Sudip Kumar Chaurasiya

College: Model Institute Of Technology(MIT)

Organization: Spurvance

Date: Oct 12, 2026

Introduction

In these four mini-projects, I practiced basic networking and cybersecurity tools. I designed a small network, captured packets with Wireshark, checked open ports with Nmap and used Python with Scapy to observe packet information. So the projects helped me connect theory with practical work.

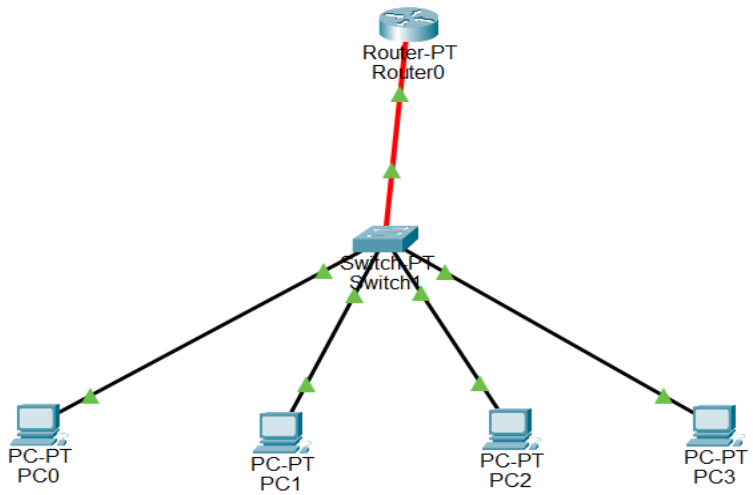
1: Small Office Network Design Using Cisco Packet Tracer

In this mini-project, I designed a small office network using Cisco Packet Tracer. I used one router, one switch, and four PCs. I connected the devices using Ethernet cables, assigned IP addresses, and configured the router interface. Finally, I tested the network connection using the ping command.

2. Network Topology

The network consists of one router, one switch, and four computers. So the switch connects all four computers while the router connects to the switch and provides a gateway for the network.

Figure 1: Small Office Network Topology



3. Network Configuration

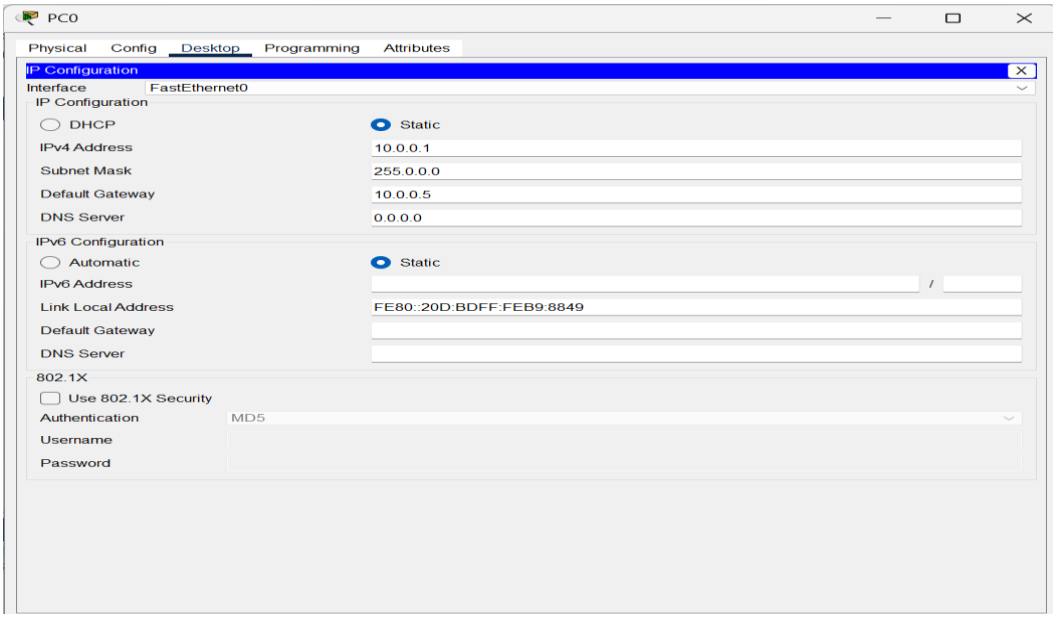
3.1 IP Address Configuration

I assigned IP addresses to the computers and router interface using the 10.0.0.0 network.

Device	Interface	IP Address	Subnet Mask
Router0	FastEthernet4/0	10.0.0.5	255.0.0.0
PC0	FastEthernet0	10.0.0.1	255.0.0.0
PC1	FastEthernet0	10.0.0.2	255.0.0.0
PC2	FastEthernet0	10.0.0.3	255.0.0.0
PC3	FastEthernet0	10.0.0.4	255.0.0.0

So the default gateway for all PCs is 10.0.0.5.

Figure 2: IP Configuration of PC0



3.2 Router Configuration

I configured the router interface using the Cisco IOS command-line interface (CLI). I assigned the IP address 10.0.0.5 and enabled the interface.

The commands used were:

Router> enable

Router# configure terminal

Router(config)# interface FastEthernet4/0

Router(config-if)# ip address 10.0.0.5 255.0.0.0

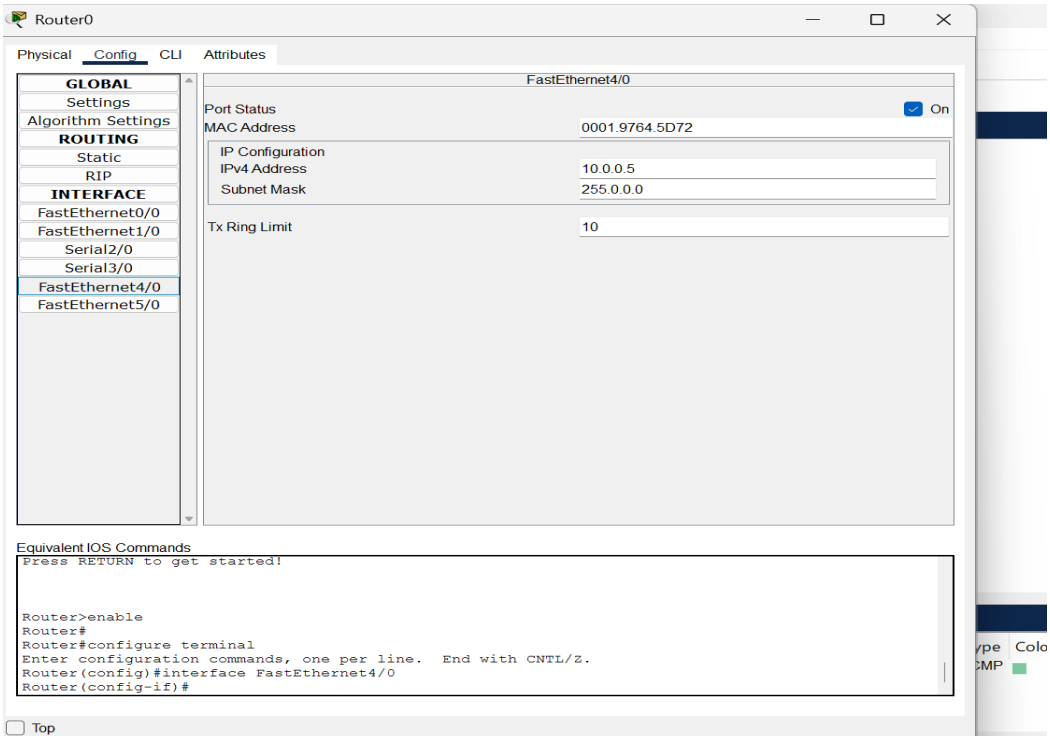
Router(config-if)# no shutdown

Router(config-if)# exit

Router(config)# end

Router# show ip interface brief

Figure 3: Router Interface Configuration



4. Connectivity Testing

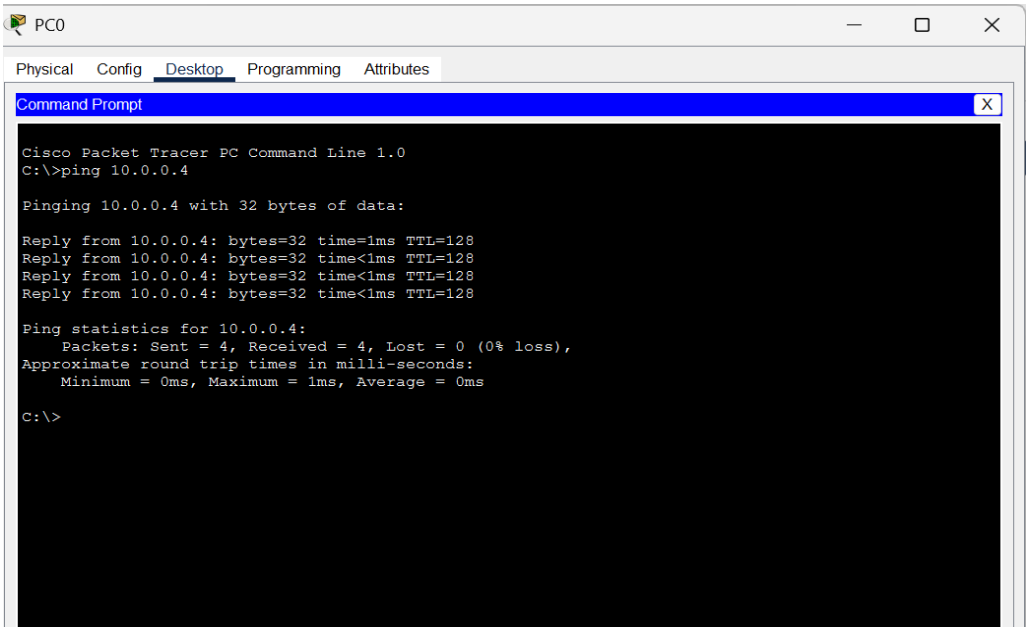
I used the ping command to check communication between PC0 and PC3.

The command was:

ping 10.0.0.4

The ping test returned four replies. Four packets were received, and no packets were lost.

Figure 4: Successful Ping Test Between PC0 and PC3



5. Results and Observations

The network devices were connected using a switch in a star topology. So the ping test between PC0 and PC3 was successful, showing that the two computers could communicate within the local area network.

The test results were:

- Packets sent: 4
- Packets received: 4

- Packets lost: 0
- Packet loss: 0%

What I learned

From this mini-project, I learned how to design a small office network using Cisco Packet Tracer. I practiced connecting network devices, assigning IP addresses, configuring a router interface and testing connectivity using the ping command. So this practical helped me understand the basic working of a local area network (LAN).

Capture and Analyze Traffic in Wireshark

2: Capture and Analyze Traffic in Wireshark

In this mini-project, I used Wireshark to capture and analyze network traffic. I checked different packets and observed their source, destination and protocols. So this helped me understand how data travels through a network.

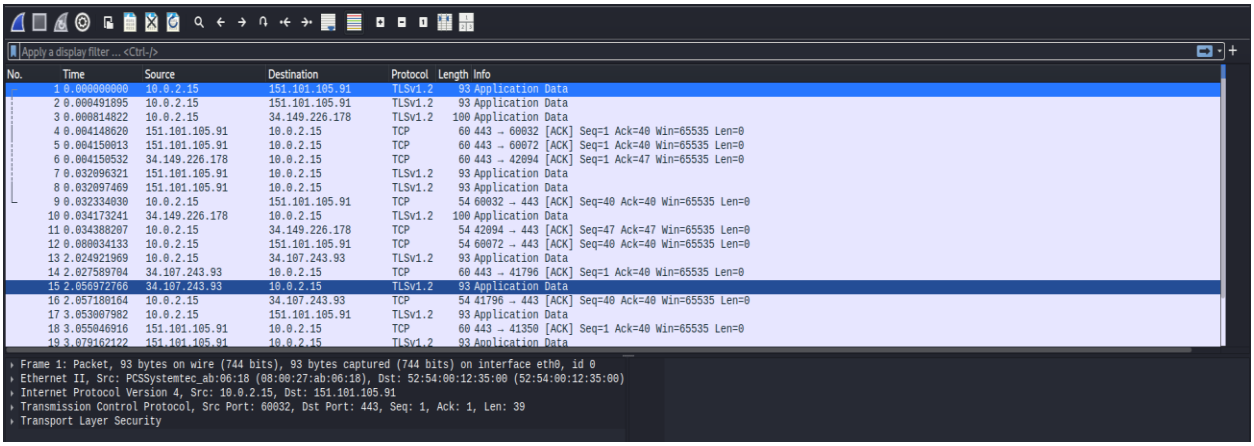
2. Steps I Followed

1. I opened Wireshark on my computer.
2. I selected my active network interface.
3. I started capturing network traffic.
4. I opened a website to generate traffic.
5. I used filters to find different packets.
6. I selected packets to check their details.

3. Traffic Capture

Wireshark showed different packets when I used the internet. I checked the protocol, source address, destination address and packet information.

Figure 1: Network traffic captured in Wireshark.



4. Traffic Analysis

I used these filters to check the packets.

DNS filter: dns

It helped me find DNS queries and responses. DNS is used to find the IP address of a domain name.

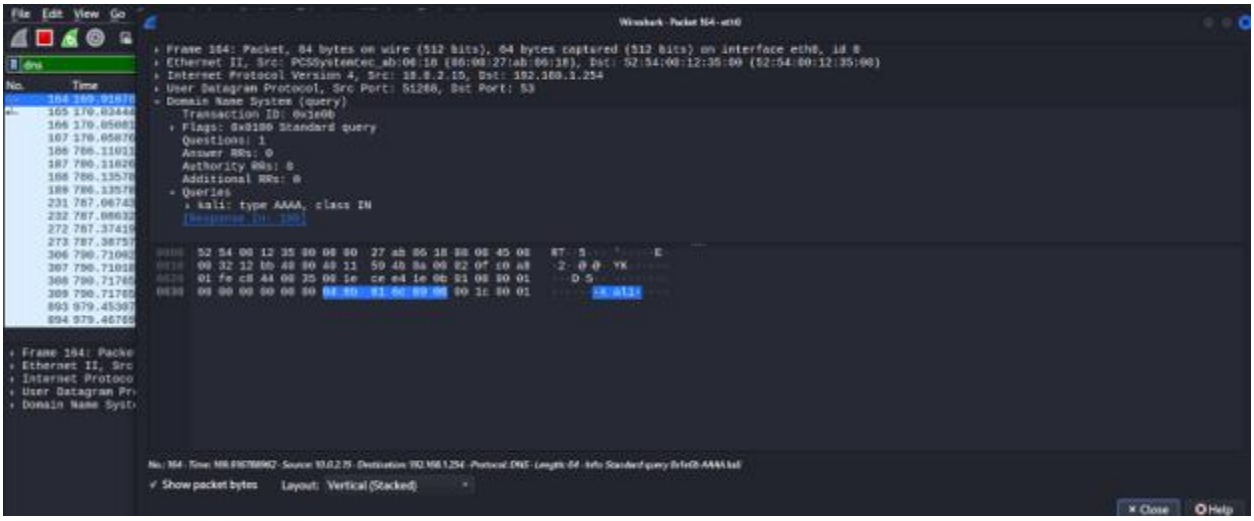
TCP filter: tcp

It helped me view TCP packets exchanged between devices.

HTTP filter: http

It helped me find HTTP requests and responses. I observed GET requests used by a browser to request website resources.

Figure 2: Packet details in Wireshark.



What I Learned

From this project, I learned how to capture network traffic using Wireshark and apply display filters. I understood source and destination addresses and learned how to identify different network protocols. I also learned how Wireshark helps analyze network communication.

Nmap Lab Scan

3: Nmap Lab Scan

In this mini-project, I used Nmap on Kali Linux to scan my Metasploitable 2 virtual machine. The main purpose was to identify open ports and running services on the target machine. I first checked the connection using the ping command and then performed a service detection scan. So this practical helped me understand network scanning and basic security assessment.

2. Tools and Lab Environment

I used the following tools for this project:

- **Kali Linux:** Used as the scanning machine.
- **Metasploitable 2:** Used as the target machine in my authorized lab.
- **Nmap:** Used to identify open ports and services.
- **Terminal:** Used to execute commands and view results.

3. Steps I Followed

Step 1: Check Connectivity

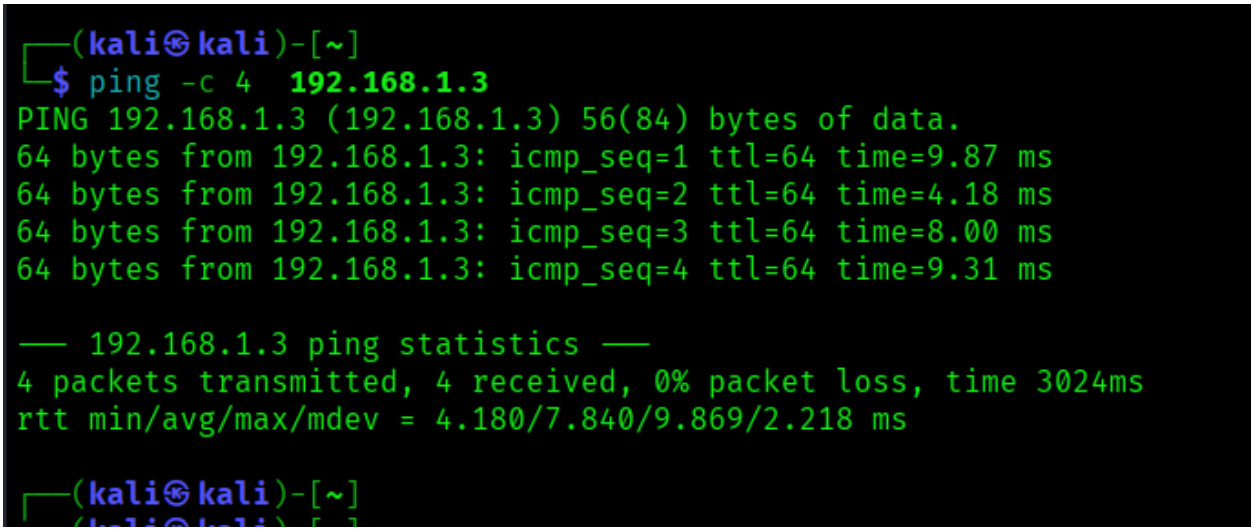
First, I checked whether the target machine was reachable from Kali Linux.

Command:

ping -c 4 192.168.1.3

I received four replies from the target machine, and the packet loss was 0%. So this confirmed that the target was reachable.

Figure 1: Ping Test of Metasploitable 2



Step 2: Perform the Nmap Scan

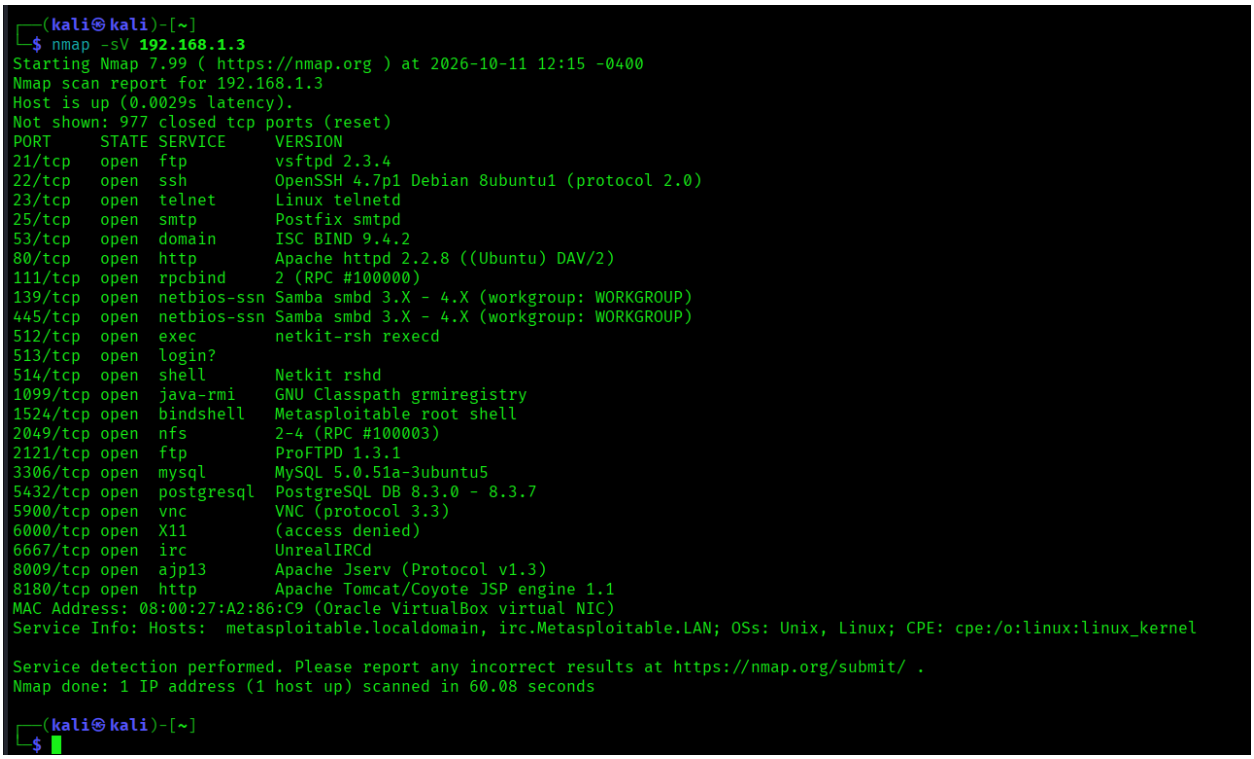
Next, I used Nmap to identify open ports and detect the services running on the target machine.

Command:

nmap -sV 192.168.1.3

The -sV option attempts to identify the services and their versions on open ports.

Figure 2: Nmap Scan of Metasploitable 2



4. Scan Results

The Nmap scan identified **23 open TCP ports** on the target machine. Some of the detected services are listed below.

Port	Service	Detected Version
21/tcp	FTP	vsftpd 2.3.4
22/tcp	SSH	OpenSSH 4.7p1
23/tcp	Telnet	Linux telnetd

25/tcp	SMTP	Postfix smtpd
53/tcp	DNS	ISC BIND 9.4.2
80/tcp	HTTP	Apache 2.2.8
139/tcp	NetBIOS	Samba
445/tcp	SMB	Samba
3306/tcp	MySQL	MySQL 5.0.51a
5432/tcp	PostgreSQL	PostgreSQL 8.3.x
5900/tcp	VNC	Protocol 3.3
6667/tcp	IRC	UnrealIRCd
8180/tcp	HTTP	Apache Tomcat

So the scan also identified services such as RPC, NFS, Java RMI and remote shell services.

5. Security Observations

During the scan, I observed the following potential security concerns:

1. **Outdated software:** Several detected services use old software versions that may contain known vulnerabilities.
2. **Telnet service:** Telnet does not encrypt communication by default, which can expose login information.
3. **Many open ports:** Multiple running services increase the attack surface of the machine.
4. **Database services:** MySQL and PostgreSQL should have strong authentication and appropriate network access restrictions.

So these observations indicate possible risks. Further testing and checking reliable security advisories would be needed to confirm specific vulnerabilities.

What I Learned

From this mini-project, I learned how to use Nmap to scan a lab machine, identify open ports and running services and detect service versions using the -sV option. I also understood the security risks of outdated services and unnecessary open ports and the importance of scanning only authorized systems.

Python packet sniffer

4: Python Packet Sniffer Using Scapy

In this mini-project, I used Python and Scapy to capture and observe network packets. Scapy is a Python library used for working with network packets. So this practical helped me understand how data travels through a network.

2. Steps I Followed

1. I opened Kali Linux.
2. I checked whether Python and Scapy were installed.
3. I created a Python file for the packet sniffer.
4. I ran the program with the required permissions.
5. I generated some network traffic and observed the captured packets.

3. Python Packet Sniffer

I used Python and Scapy to capture packets from my network interface. The program displayed basic information about the packets, such as their source and destination addresses.

Code used:

```
Session  Actions  Edit  View  Help
  GNU nano 9.2                                                  sniffer.py
from scapy.all import sniff, IP

def show_packet(packet):
    if IP in packet:
        source = packet[IP].src
        destination = packet[IP].dst
        protocol = packet[IP].proto

        print("Source IP:", source)
        print("Destination IP:", destination)
        print("Protocol:", protocol)
        print("-" * 30)

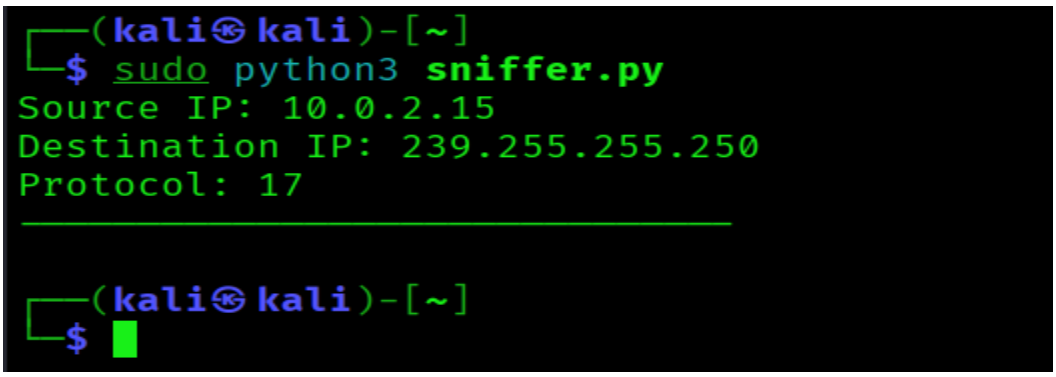
sniff(prn=show_packet, count=10)
```

So this program captures 10 packets and displays their source IP, destination IP, and IP protocol number when available.

4. Practical Result

After running the program, I observed packet information in the terminal. The source and destination IP addresses helped me understand where the packets were coming from and where they were going.

Figure 1: Python packet sniffer running in Kali Linux.



What I Learned

I learned the basics of Scapy in Python and how to capture network packets. I also understood source and destination IP addresses and how Python can be used for network traffic analysis.

Overall Conclusion

Through these four mini-projects, I practiced designing a small network, capturing and analyzing packets, scanning an authorized lab machine and writing a basic packet sniffer with Python and Scapy. I learned how IP addresses, ports, protocols and packet details help explain network communication. So these practical tasks improved my basic networking and cybersecurity skills. I will add my own screenshots and record actual results before submitting the report.